

《最后的要塞（塔防游戏）》

详细设计

OKK 项目组

2026 年 07 月 08 日

版本修订记录

版本标识	注 释	作 者	日 期
1.0	初始版本	开发组全体	2026/07/08
1.1			
1.2			
1.3			

一、整体系统模块设计

1. 模块概述

本项目为 C++Qt6 开发的 Windows 单机 2D 卡牌塔防游戏《最后的要塞》，整体采用五层分层架构，包含底层资源层、场景地图层、战斗逻辑层、特色增益层、UI 交互层。所有模块低耦合拆分，实现塔防完整游玩闭环，核心实现造塔防守、怪物波次进攻、子弹碰撞、卡牌永久 Buff 强化、游戏状态控制、结算展示功能，满足课程设计交付标准，无网络、联机、付费逻辑，全部资源本地内置。

2. 系统架构



图 1.1.1 系统架构图

3. ER 图

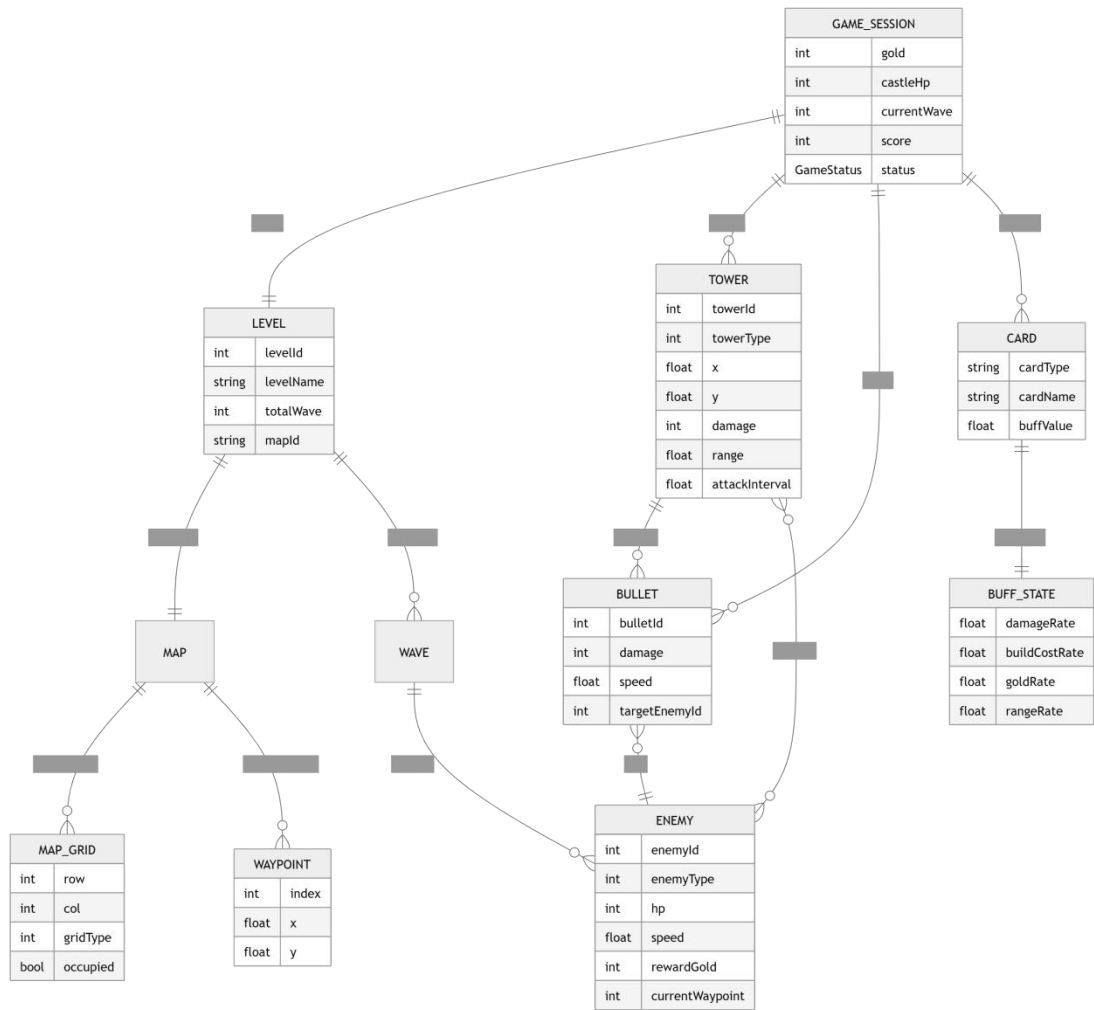


图 1.1.2 游戏核心实体关系图

4. 全局通用状态码设计状态码

(1) 全局业务状态码

表 1.4.1			
code	message	游戏弹窗提示文本	说明
200	操作成功	无提示	所有模块正常执行返回
500001	素材资源加载失败	图片资源缺失，部分画面无法正常显示	底层资源读取 qrc 素材异常
500002	游戏对象空指针异常	游戏运行异常	塔、怪物、子弹对象未初始化 / 已销毁仍调用
500003	画面窗口渲染异常	画面渲染出错	窗口缩放、画布重绘失败
400001	金币不足	金币不足，无法建造该防御塔	造塔时金币数值校验不通过
400002	建造区域非法	该区域禁止建造防御塔	点击路径、要塞禁区造塔
400003	游戏状态非法	当前状态无法执行该操作	暂停 / 卡牌弹窗阶段执行造塔、波次逻辑
400004	数值越界异常	数值超出合法范围	血量、金币、波次出现负数 / 超大值

(2) 游戏内部状态枚举（全局 GameStatus）

表 1.4.2		
枚举值	注释	触发场景
MENU	主菜单界面	程序启动、返回菜单
RUNNING	战斗正常运行	对局正常进行，循环刷新单位逻辑
PAUSE	游戏暂停	空格键触发暂停
CARD_POPUP	卡牌弹窗开启	单波怪物全部清除，弹出卡牌选择
WIN_SETTLE	胜利结算界面	全部波次通关
LOSE_SETTLE	失败结算界面	要塞血量归零

二、底层资源模块详细设计

1. 模块概述

游戏最底层基础依赖模块，统一管理 qrc 内置图片、音效素材加载、内存缓存、资源销毁，提供全局统一素材读取接口，解决重复加载、内存泄漏、素材丢失问题，上层地图、塔、怪物、UI 全部依赖本模块获取资源。

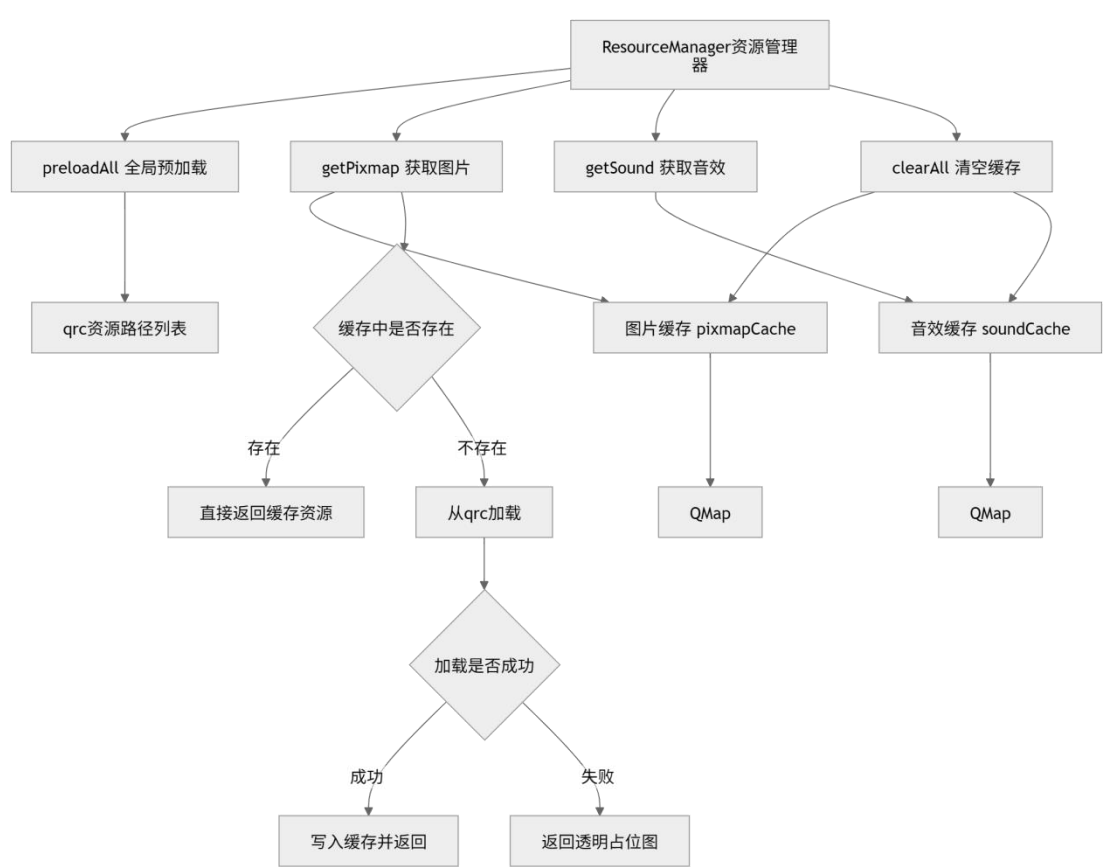


图 2.1.1 模块结构设计图

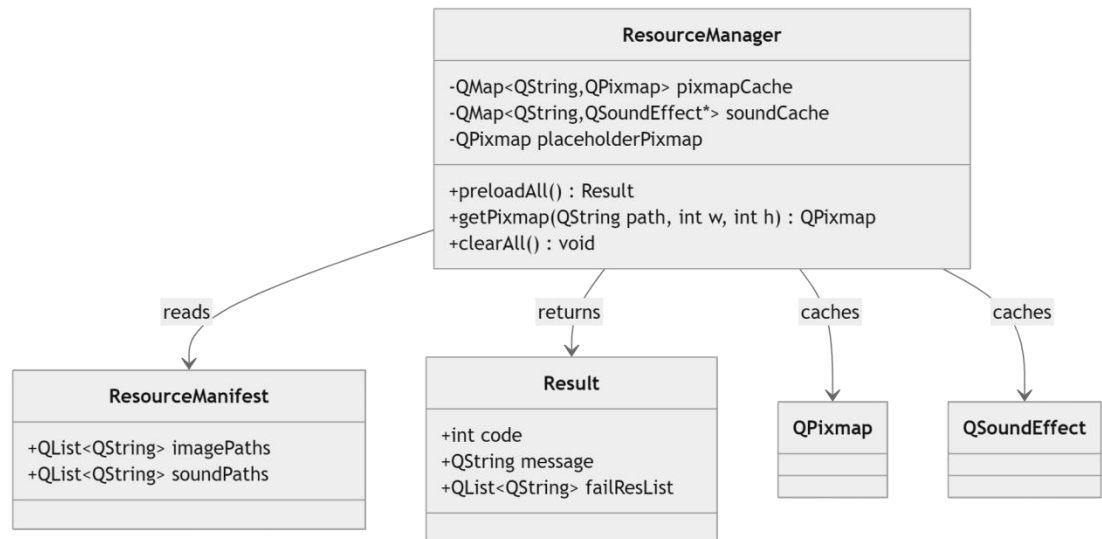


图 2.1.2 对象关系图

2. 内部数据结构（内存缓存结构）

```
1 struct ResourceResult {
2     int code = 200;
3     QString message = "操作成功";
4     QList<QString> failResList;
5 };
6
7 struct QPixmapCacheItem {
8     QString resPath;
9     QPixmap pixmap;
10    int width = 0;
11    int height = 0;
12    bool loaded = false;
13 };
14
15 class ResourceManager {
16 public:
17     ResourceResult preloadAll();
18     QPixmap getPixmap(const QString& resPath, int targetW = 0, int targetH = 0);
19     void clearAll();
20
21 private:
22     QMap<QString, QPixmap> pixmapCache;
23     QMap<QString, QSoundEffect*> soundCache;
24     QPixmap placeholderPixmap;
25
26 private:
27     QString makePixmapKey(const QString& resPath, int targetW, int targetH) const {
28         return resPath + "#" + QString::number(targetW) + "x" + QString::number(targetH);
29     }
30 };
```

图 2.2.1

3. 接口设计

(1) 状态码

复用全局业务状态码 200、500001、500003

(2) 接口 1：全局预加载全部素材

接口用途：程序启动时一次性加载所有本地 qrc

素材接口说明：主线程同步执行，加载失败记录失败素材列表，
不阻塞程序启动

支持描述：单线程同步执行

接口标识：res/v1/preloadAll

返回类型及调用方式：同步调用，结构体返回

(3) 参数说明

表 2.3.1

返回参数：			
字段	注释	字段类型	是否必须
code	状态码	int	是
message	加载结果提示	string	是
fail_res_list	加载失败素材路径集合	QList<QString>	否

(4) 接口 2：获取图片素材

表 2.3.4

请求参数：					
字段	注释	字段类型	是否必须	范围	备注
res_path	qrc 素材路径	string	是	1~256	本地内置资源地址
target_w	图片缩放宽度	int	否	大于 0	不传返回原图尺寸
target_h	图片缩放高度	int	否	大于 0	不传返回原图尺寸
GET 返回参数：QPixmap 素材对象，缺失返回空白透明占位图					

(5) 接口 3：清空全部资源缓存

接口标识：res/v1/clearAll

触发时机：退出游戏、重新开局、返回主菜单

逻辑：清空图片、音效缓存 Map，销毁所有素材实例，释放堆内存

三、场景地图模块详细设计

1. 模块概述

负责游戏战斗地图渲染、敌人移动路径点位存储、鼠标点击坐标建造区域校验，内置单张固定中世纪地图，区分可建造空地、行走路径、要塞禁区，支持窗口自适应缩放，为造塔逻辑提供坐标校验能力。

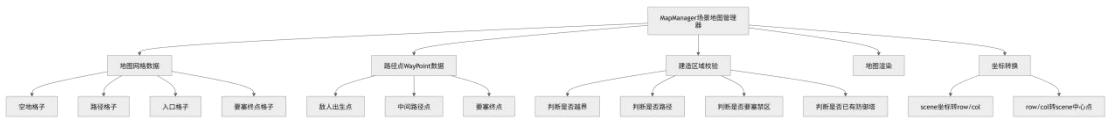


图 3.1.1 模块结构设计图

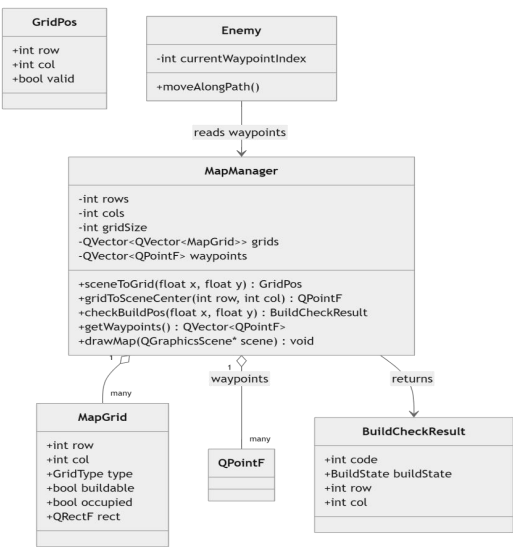


图 3.1.2 对象关系图

2. 内部存储结构

```
enum class GridType {
    Empty = 0,      // 普通空地
    Road = 1,       // 敌人路径
    Castle = 2,     // 要塞终点
    Entrance = 3    // 敌人入口
};

enum class BuildState {
    CanBuild = 1,
    RoadForbidden = 2,
    CastleForbidden = 3,
    Occupied = 4,
    OutOfMap = 5
};

struct GridPos {
    int row = -1;
    int col = -1;
    bool valid = false;
};

struct MapGrid {
    int row = 0;
    int col = 0;
    GridType type = GridType::Empty;
    bool buildable = true;
    bool occupied = false;
    QRectF rect;
};

struct BuildCheckResult {
    int code = 200;
    BuildState buildState = BuildState::CanBuild;
    int row = -1;
    int col = -1;
};
```

图 3.2.1

3. 接口设计

(1) 状态码

复用全局状态码 200、400002、500003

(2) 接口：坐标建造区域校验

接口用途：校验鼠标点击坐标是否允许建造防御塔

接口说明：传入鼠标点击 XY 坐标，返回建造许可枚举

支持描述：单线程帧循环同步调用

接口标识：map/v1/checkBuildPos

调用方式：同步调用

(3) 参数说明

表 3.3.1

请求参数:						
字段	注释	字段类型	是否必须	范围	备注	
x	鼠标点击 X 坐标	float	是	0~ 窗口宽度	画布像素坐标	
y	鼠标点击 Y 坐标	float	是	0~ 窗口宽度	画布像素坐标	
GET 返回参数:						
字段	注释		字段类型	是否必须	可选值	
code	状态码		int	是	200/400002	
build_state	建造状态枚举		int	是	1 = 允许建造 2 = 路径禁区 3 = 要塞禁区	

四、战斗单位模块详细设计

1. 模块概述

核心战斗逻辑模块，分为防御塔管理、怪物敌人管理、子弹碰撞检测三个子模块，由全局 QTimer 战斗主循环每帧驱动刷新，处理单位移动、攻击冷却、子弹生成、碰撞命中、对象销毁、金币奖励结算全流程，为游戏核心 P0 必实现功能。



图 4.1.1 模块结构设计图

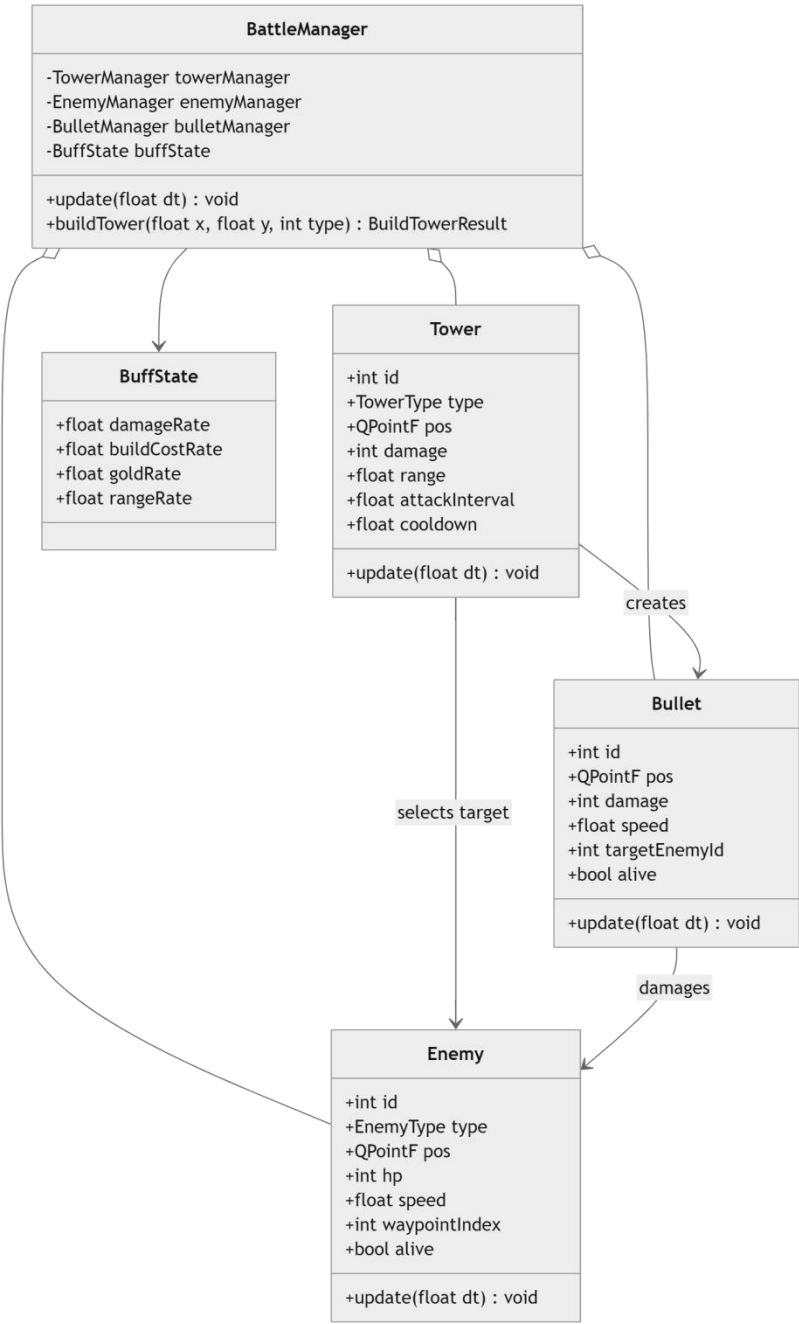


图 4.1.2 ER 对象关系图

2. 核心对象内存结构体

```
enum class TowerType {
    Archer = 1,
    Mage = 2
};

enum class EnemyType {
    Goblin = 1,
    Orc = 2,
    Knight = 3
};

struct TowerObject {
    int towerId = 0;
    TowerType type = TowerType::Archer;

    QPointF pos;
    int gridRow = 0;
    int gridCol = 0;

    int damage = 10;
    float attackRange = 150.0f;
    float attackInterval = 1.0f;
    float cooldown = 0.0f;

    int cost = 50;
    bool active = true;
};
```

```
struct EnemyObject {
    int enemyId = 0;
    EnemyType type = EnemyType::Goblin;

    QPointF pos;
    int maxHp = 100;
    int currentHp = 100;

    float speed = 60.0f;
    int currentWaypointIndex = 0;

    int rewardGold = 10;
    int scoreValue = 10;
    bool alive = true;
    bool reachedCastle = false;
};

struct BulletObject {
    int bulletId = 0;

    QPointF pos;
    QPointF velocity;

    int damage = 10;
    float speed = 300.0f;
    int targetEnemyId = -1;

    bool active = true;
};
```

图 4.2.1

3. 接口设计（核心造塔接口）

(1) 状态码

全局复用：200、400001、400002、400003、500002

(2) 接口：建造防御塔

接口用途：校验金币、坐标，扣除金币生成塔对象

接口说明：玩家点击空地后调用，校验双重条件（区域允许 + 金币充足）

支持描述：单线程同步帧调用

接口标识：battle/tower/v1/buildTower

请求方式：同步调用

(3) 参数说明

表 4.3.1

请求参数：					
字段	注释	字段类型	是否必须	范围	备注
x	建造坐标 X	float	是	画布坐标	鼠标点击位置
y	建造坐标 Y	float	是	画布坐标	鼠标点击位置
tower_type	塔类型	int	是	1/2	1 弓箭塔, 2 法师塔
GET 返回参数：					
字段	注释	字段类型	是否必须		
code	状态码	int	是		
message	提示文本	string	是		
tower_id	新建塔唯一标识	int	否		

五、卡牌强化特色模块详细设计

1. 模块概述

本局永久 Buff 增益特色模块，每波怪物全部清除后自动触发弹窗，随机生成 3 张不重复强化卡牌，玩家选择后全局属性数值永久

叠加，包含伤害提升、建造降价、金币加成、攻击范围扩大四类增益，独立低耦合模块，不侵入基础战斗逻辑。

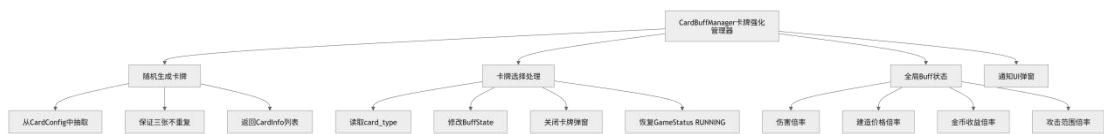


图 5.1.1 模块结构设计图

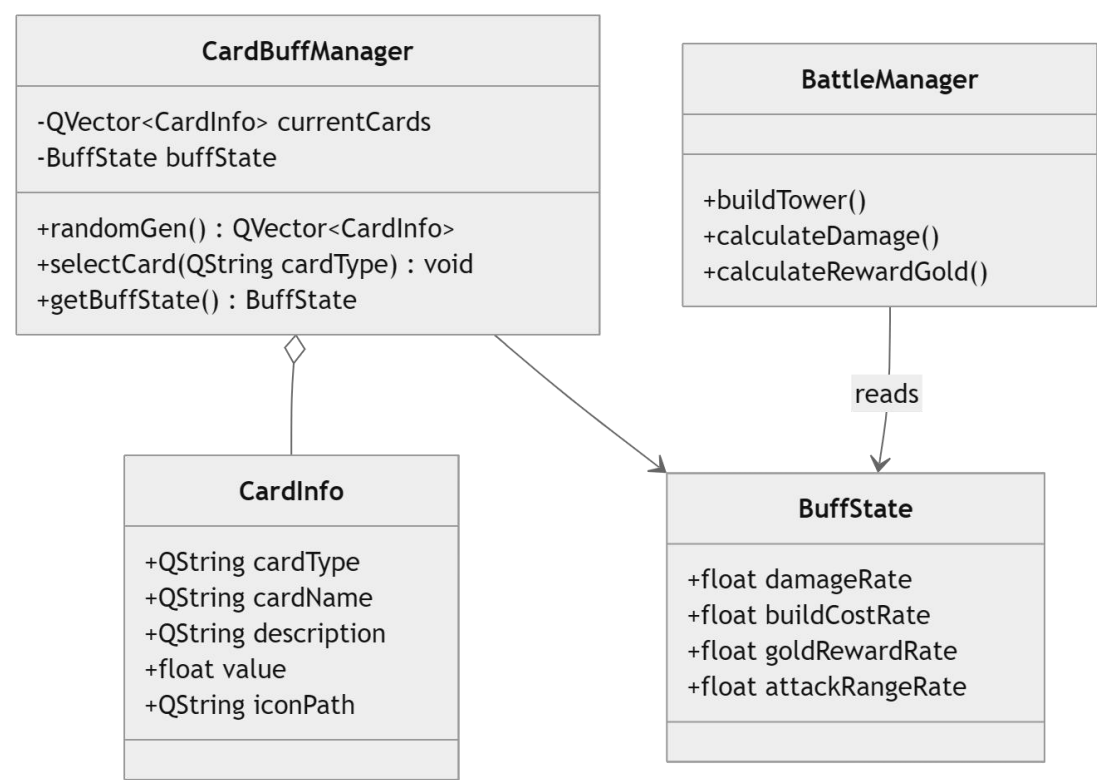


图 5.1.2 对象关系图

2. 存储结构（全局叠加 Buff）

```
enum class CardType {
    DamageUp,
    BuildCostDown,
    GoldRewardUp,
    RangeUp
};

struct CardInfo {
    CardType type;
    QString name;
    QString description;
    float value = 0.0f;
    QString iconPath;
};

struct BuffState {
    float damageRate = 1.0f; // 伤害倍率
    float buildCostRate = 1.0f; // 建造价格倍率，小于1表示降价
    float goldRewardRate = 1.0f; // 金币奖励倍率
    float attackRangeRate = 1.0f; // 攻击范围倍率
};

inline void applyCardBuff(BuffState& buff, CardType type) {
    switch (type) {
        case CardType::DamageUp:
            buff.damageRate += 0.15f;
            break;
        case CardType::BuildCostDown:
            buff.buildCostRate *= 0.90f;
            break;
        case CardType::GoldRewardUp:
            buff.goldRewardRate += 0.20f;
            break;
        case CardType::RangeUp:
            buff.attackRangeRate += 0.10f;
            break;
    }
}
```

图 5.2.1

3. 接口设计

(1) 接口：随机生成卡牌列表

接口标识：buff/card/v1/randomGen

用途：每波清理完成生成 3 种不同增益卡牌返回卡牌类型数组，
供弹窗渲染图片

(2) 接口：选中卡牌叠加 Buff

接口标识：buff/card/v1/selectCard

请求参数 card_type：卡牌类型字符串

执行逻辑：对应倍率叠加，关闭弹窗，恢复游戏 RUNNING 状态

六、UI 界面系统模块详细设计

1. 模块概述

统一管理游戏全部可视化界面，包含主菜单、战斗顶部 HUD 面板、卡牌模态弹窗、胜利 / 失败结算弹窗，处理所有鼠标、键盘交互事件，实时同步展示金币、血量、波次、分数数值。

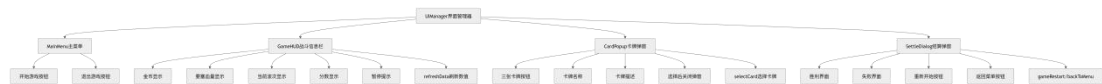


图 6.1.1 模块结构设计图

2. 接口设计

(1) 接口：全局数值刷新 UI

接口标识：ui/hud/v1/refreshData

请求参数：当前金币、要塞血量、当前波次、总分数

作用：实时更新战斗顶部 HUD 面板文本

七、全局游戏状态控制模块详细设计

1. 模块概述

统一管控游戏全局运行状态，实现暂停 / 继续、重新开局、返回菜单、退出程序逻辑，全局唯一 `GameStatus` 枚举，战斗主循环每帧优先读取状态，非 `RUNNING` 状态跳过所有战斗逻辑，节省 CPU 性能。

2. 核心接口

1. `togglePause ()`: 空格键切换 `RUNNING/PAUSE` 状态

2. `gameRestart ()`: 清空所有战斗对象，重置全局数值，重新开局

3. `backToMenu ()`: 销毁战斗场景，释放资源，切换至主菜单

4. `exitGame ()`: 释放全部素材、销毁所有堆对象，关闭程序进程

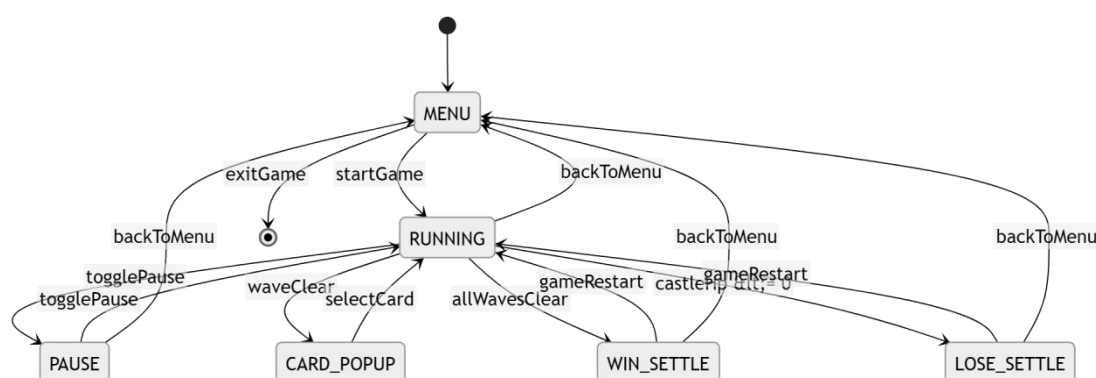


图 7.2.1 流程图

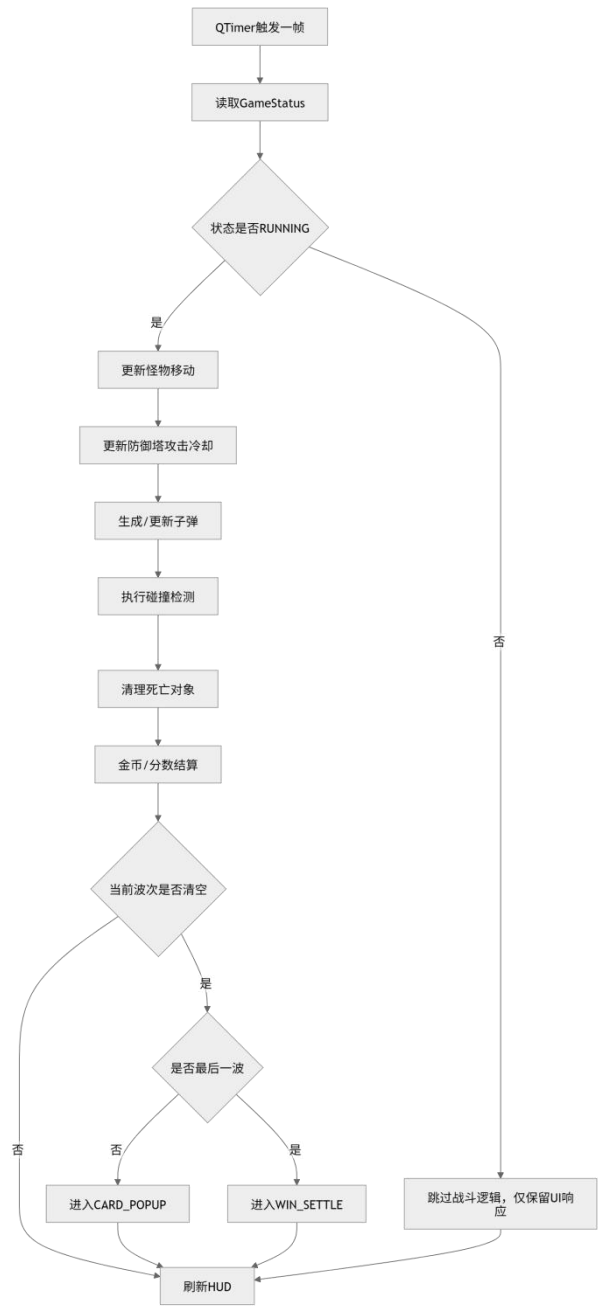


图 7.2.2 流程图

八、性能、内存、异常统一规范

1.帧率约束：战斗主循环锁定 30FPS，单帧执行耗时 $\leq 10\text{ms}$ ，50个单位同时存在无卡顿

2. 内存约束：所有塔、怪物、子弹动态堆对象销毁前置空指针，切换界面统一释放素材缓存，杜绝内存泄漏

3. 数值容错：金币、血量、倍率全部下限 0，防止负数、数值溢出

4. 异常防护：所有对象访问前做空指针判断，素材缺失返回空白占位，不崩溃程序